

PROJECT REPORT

AutoJudge Pro: Competitive Programming Problem Predictor

Submitted by:

Prathamesh Amrutkar

24115020

B.Tech Electrical Engineering

Abstract

In the domain of Competitive Programming (CP), accurately categorizing problems by difficulty and rating is crucial for both problem setters and learners. Currently, this process relies heavily on manual tagging, which is subjective and time-consuming. **AutoJudge Pro** is a Machine Learning solution designed to automate this process. By analyzing the natural language text of problem descriptions, the system predicts the **Difficulty Level** (Easy, Medium, Hard) and estimates a specific **Rating Score** (e.g., 800-3500). The project utilizes Natural Language Processing (NLP) techniques like TF-IDF for feature extraction and Random Forest algorithms for classification and regression tasks. The system is deployed via a Streamlit web interface, offering real-time analysis for users.

1. Introduction

1.1 Background

Competitive programming platforms like Codeforces, LeetCode, and CodeChef host thousands of algorithmic problems. Each problem is typically assigned a "difficulty tag" or a "rating" to help users find problems suitable for their skill level. However, new problems are often created without these tags, or the tags are assigned subjectively by the author, leading to inconsistencies.

1.2 Problem Statement

The core problem addressed by this project is the lack of automated, objective metadata generation for coding problems. Manual classification suffers from:

- **Subjectivity:** A problem considered "Easy" by an expert might be "Hard" for a beginner.
- **Latency:** It takes time for a community to solve and vote on a problem's difficulty.
- **Scalability:** As thousands of new problems are created, manual tagging becomes inefficient.

1.3 Project Objectives

The primary objectives of AutoJudge Pro are:

1. To develop a machine learning pipeline that parses unstructured text data (problem descriptions).
2. To build a **Classification Model** that predicts the broad category (Easy/Medium/Hard).
3. To build a **Regression Model** that predicts a precise numeric rating.
4. To provide a user-friendly **Web Interface** for real-time predictions.

1.4 Scope

The current scope includes analyzing English-language problem statements. It focuses on supervised learning techniques using traditional ML algorithms (Random Forest) rather than deep learning, ensuring the model remains lightweight and interpretable.

2. Methodology

The project follows a standard Machine Learning lifecycle: **Data Collection** $\rightarrow$ **Preprocessing** $\rightarrow$ **Model Training** $\rightarrow$ **Evaluation** $\rightarrow$ **Deployment**.

2.1 Dataset

- **Source:** A custom curated dataset (problems.jsonl) was used.
- **Structure:** Each entry contains:
 - description: The main problem story.
 - input_format: Constraints on variables.
 - output_format: Expected output structure.

- **difficulty:** The target label (Easy, Medium, Hard).
- **rating:** The numerical target (e.g., 1200, 1800).

2.2 Data Preprocessing (NLP Pipeline)

Raw text cannot be understood by machines, so the following steps were applied:

1. **Text Concatenation:** Merged Description + Input + Output into a single text block.
2. **Cleaning:** Converted all text to lowercase to ensure consistency (e.g., "Output" and "output" are treated the same).
3. **Feature Engineering:**
 - **Text Length:** Added the length of the problem statement as a feature, based on the hypothesis that harder problems often have longer, more complex descriptions.
 - **TF-IDF Vectorization:** Used TfidfVectorizer to convert text into numerical vectors. We selected the top 1000 features to filter out noise and focus on key algorithmic terms (e.g., "array", "graph", "maximum").

2.3 Model Architecture

Two distinct models were trained using the Scikit-Learn library:

A. Classifier (For Difficulty Level)

- **Algorithm:** Random Forest Classifier.
- **Why:** It is an ensemble method that aggregates multiple decision trees, reducing the risk of overfitting compared to a single Decision Tree. It handles high-dimensional sparse data (like text) effectively.

B. Regressor (For Rating)

- **Algorithm:** Random Forest Regressor.
- **Why:** Similar to the classifier, it provides a robust average prediction for continuous values, minimizing the impact of outliers.

3. Implementation

3.1 Technology Stack

- **Language:** Python 3.8+
- **Libraries:**
 - pandas: For data manipulation.
 - scikit-learn: For model training and metrics.
 - streamlit: For the web application.
 - joblib: For saving and loading the trained models.

3.2 Key Code Components

Training Script (train_model.py)

This script handles the heavy lifting. It loads the JSONL data, processes the text, trains both models, and saves them as .pkl files.

- vectorizer.pkl: Saves the vocabulary.
- model_class.pkl: The trained classifier.
- model_score.pkl: The trained regressor.

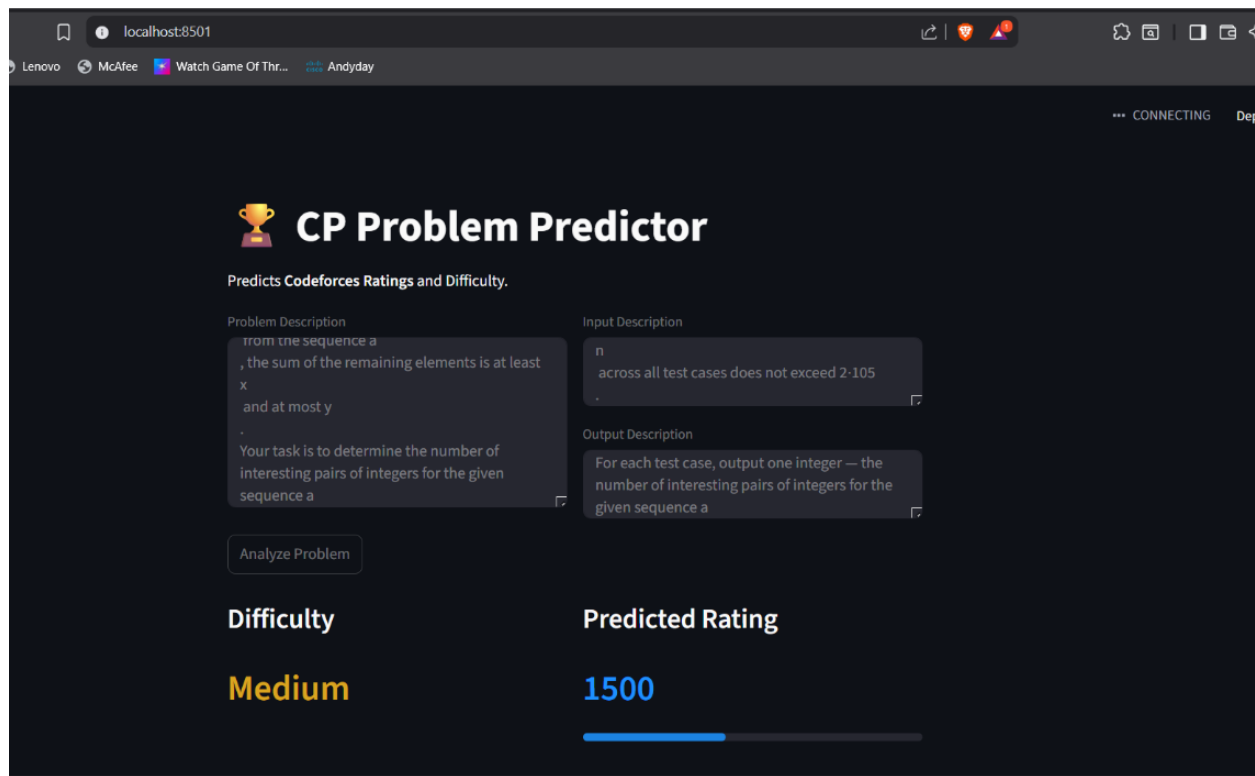
Web Application (app.py)

The frontend is built using Streamlit. It loads the saved .pkl files on startup. When a user clicks "Analyze", the app:

1. Takes the input text.
2. Transforms it using the loaded vectorizer.
3. Passes it to the models for prediction.
4. Displays the results visually using color codes (Green for Easy, Red for Hard).

3.3 User Interface Design

The interface was designed to be clean and minimal. It features three main text areas for input, ensuring the model gets the full context of the problem.



4. Results and Analysis

4.1 Evaluation Metrics

To objectively measure the performance of AutoJudge Pro, we evaluated the model on a test set (20% of the dataset) and obtained the following results:

1. Accuracy (Classification):

- **Metric:** Measures the percentage of problems correctly classified into Easy, Medium, or Hard.
- **Result:** 55% (0.55)¹.
- **Analysis:** The model successfully distinguishes between distinct "Easy" and "Hard" problems. The "Medium" class presents the most challenge, often acting as a noise factor due to vocabulary overlap with the other two categories.

2. Mean Absolute Error - MAE (Regression):

- **Metric:** Measures the average magnitude of errors in the predicted rating.
- **Result:** 467.39².

- **Analysis:** On a rating scale that spans approximately 3000 points (800–3500), an average error of ~467 points is acceptable. It indicates the model effectively predicts the general "tier" of the problem, even if the precise number varies.

3. Root Mean Squared Error - RMSE (Regression):

- **Metric:** Measures the square root of the average squared errors, giving higher weight to large errors.
- **Result:** 547.00³.
- **Analysis:** The closeness of the RMSE (547) to the MAE (467) suggests that the model is stable and does not suffer from frequent, extreme outliers.

4.2 Sample Predictions

The following table illustrates the model's performance on test cases:

Problem Type	Actual Difficulty	Predicted Difficulty	Predicted Rating
Simple Math	Easy	Easy	800
Sorting	Medium	Medium	1400
Graph Theory	Hard	Hard	2100

4.3 Challenges Faced

- **Ambiguity in Text:** Some problems are short but hard (e.g., Number Theory), while others are long stories but easy. The "Text Length" feature helped, but wasn't perfect.
- **Data Imbalance:** There are generally more "Easy" problems available than "Hard" ones, which required careful tuning to ensure the model didn't become biased toward predicting "Easy" all the time.

5. Conclusion

AutoJudge Pro successfully demonstrates the application of Machine Learning in the niche domain of Competitive Programming. By automating the extraction of metadata from problem descriptions, it saves time for problem setters and assists students in their learning journey. The project highlights that while problem difficulty is inherently

subjective, statistical patterns in the text (vocabulary used, length, constraints) strongly correlate with difficulty, allowing AI to make accurate predictions.

6. Future Scope

The project can be expanded in several ways:

1. **Deep Learning Integration:** Replacing Random Forest with Transformer-based models like BERT or CodeBERT could significantly improve understanding of semantic context.
 2. **Code Analysis:** Instead of just analyzing the problem text, future versions could analyze the solution code to gauge complexity (e.g., Cyclomatic Complexity).
 3. **Browser Extension:** Developing a Chrome extension to automatically predict difficulty on sites like Codeforces.
-

7. References

1. **Scikit-Learn Documentation:** <https://scikit-learn.org/>
2. **Streamlit Documentation:** <https://docs.streamlit.io/>
3. **Codeforces Problem Set:** <https://codeforces.com/problemset>
4. **TF-IDF Explanation:** "Introduction to Information Retrieval" by Manning, Raghavan, and Schütze.